Aula 9

Análise de Algoritmos

Cris Sato

Vamos voltar ao problema de ordenação

Entrada: um vetor de inteiros A[1...n]

Objetivo: modificar A de modo que as suas entradas fiquem em ordem crescente

Vimos até agora os seguintes algoritmos:

- SelectionSort
- InsertionSort
- MergeSort
- QuickSort

Vamos ver agora o HeapSort

- O HeapSort é baseado na estrutura de dados Heap, que implementa uma **fila de prioridade**
- Uma **fila de prioridade** é uma estrutura para inserção e remoção de itens de acordo com o valor de sua prioridade

- Vamos ver primeiramente um **MIN-HEAP**

MIN-HEAP

CRIA(N): cria um heap com capacidade de armazenamento N. Retorna o heap criado

INSERE(v): insere um elemento com valor v

REMOVE(): remove o elemento de **menor** valor

Em geral, também incluímos métodos para a atualização de valores

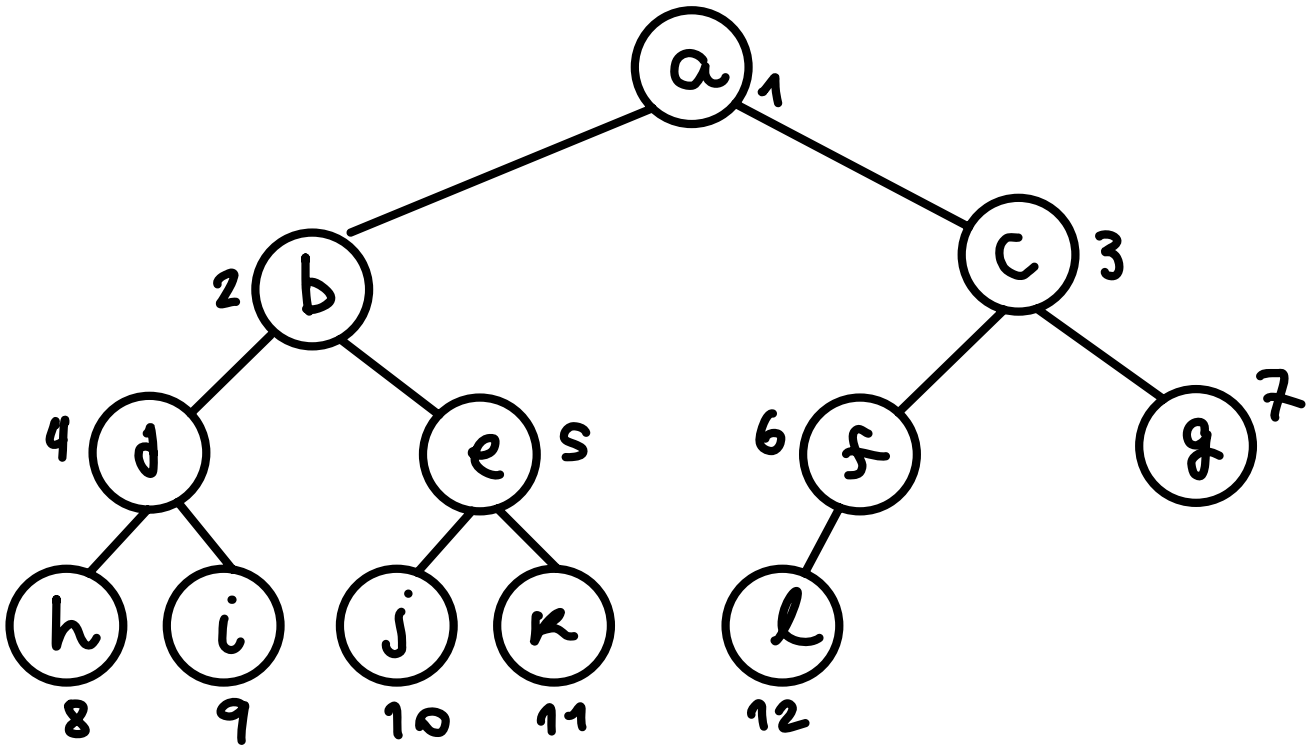
Como podemos utilizar um MIN-HEAP para ordenação?

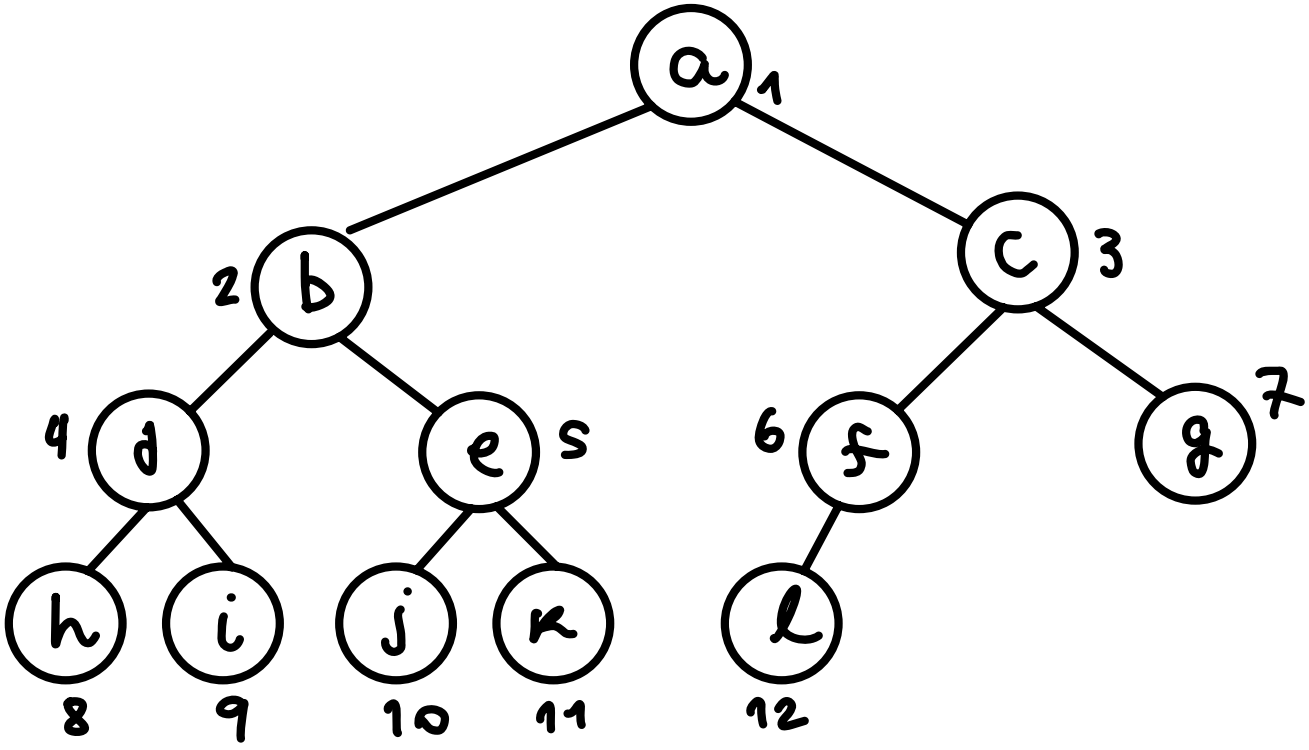
- Inserir cada elemento de A no heap
- Remover os itens um por um. Como eles estão em um MIN-HEAP, eles são retirados do menor para o maior

- O HeapSort não usa um heap separadamente, implementamos o heap no próprio vetor A, dessa forma a ordenação é feita in place

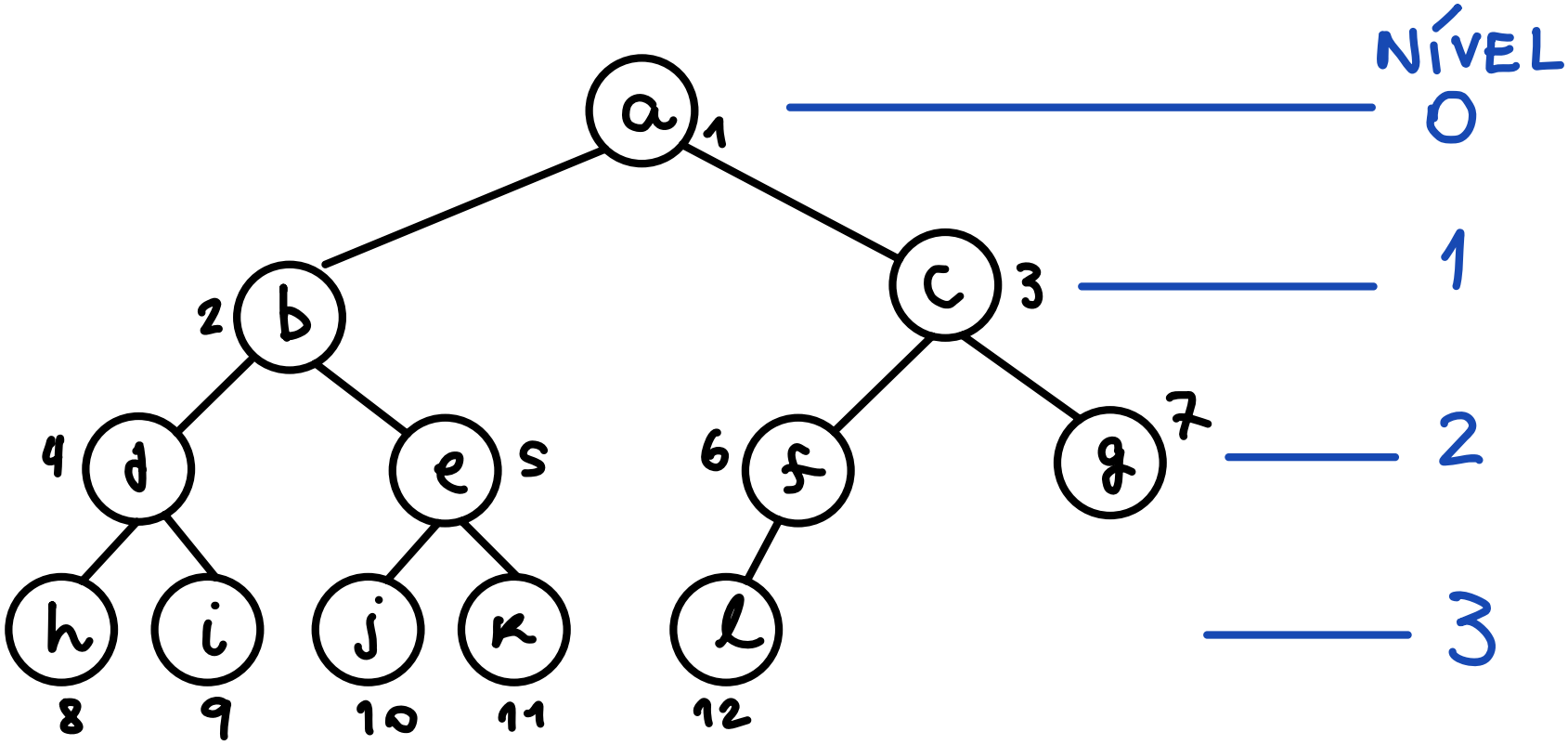
- Um heap é uma **árvore binária**
- Vamos ver uma implementação em um **vetor**
- Cada nó é **pai** de até dois filhos:
  filho esquerdo e **filho direito**

	1	2	3	4	5	6	7	8	9	10	11	n 12
A	a	b	c	d	e	f	g	h	i	j	k	l

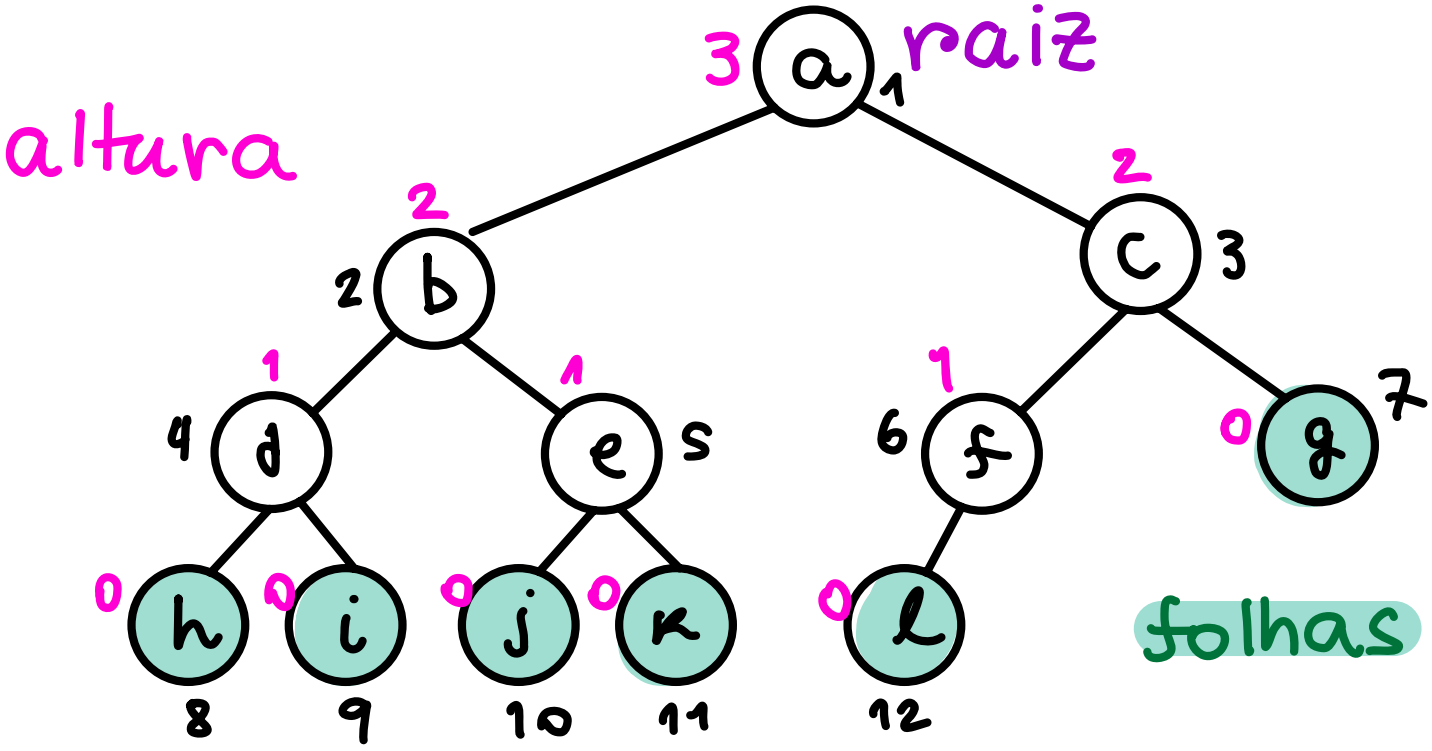




- O pai do elemento na posição i é $\lfloor i/2 \rfloor$
- O filho esquerdo de i fica na posição 2i e o filho direito de i fica na posição 2i+1

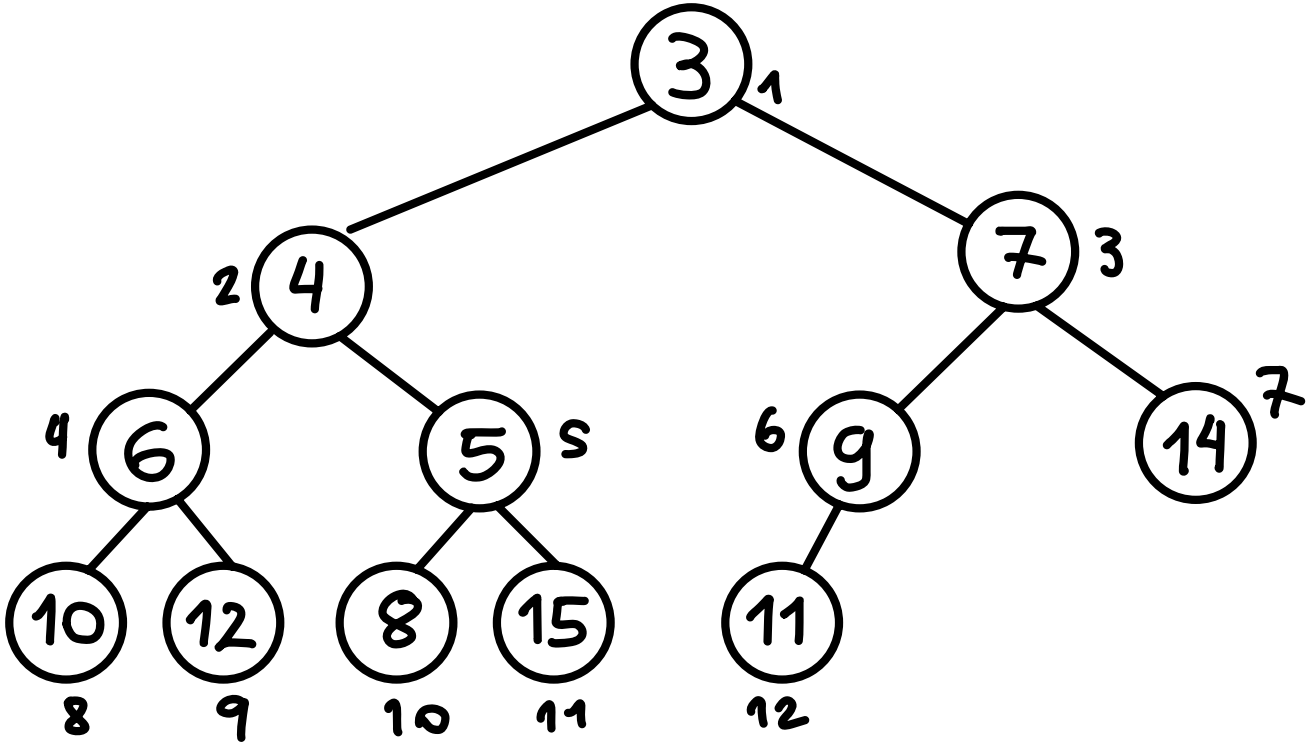


- O vértice i fica no nível $\lfloor \lg(i) \rfloor$
- O número de níveis é $1 + \lfloor \lg(n) \rfloor$

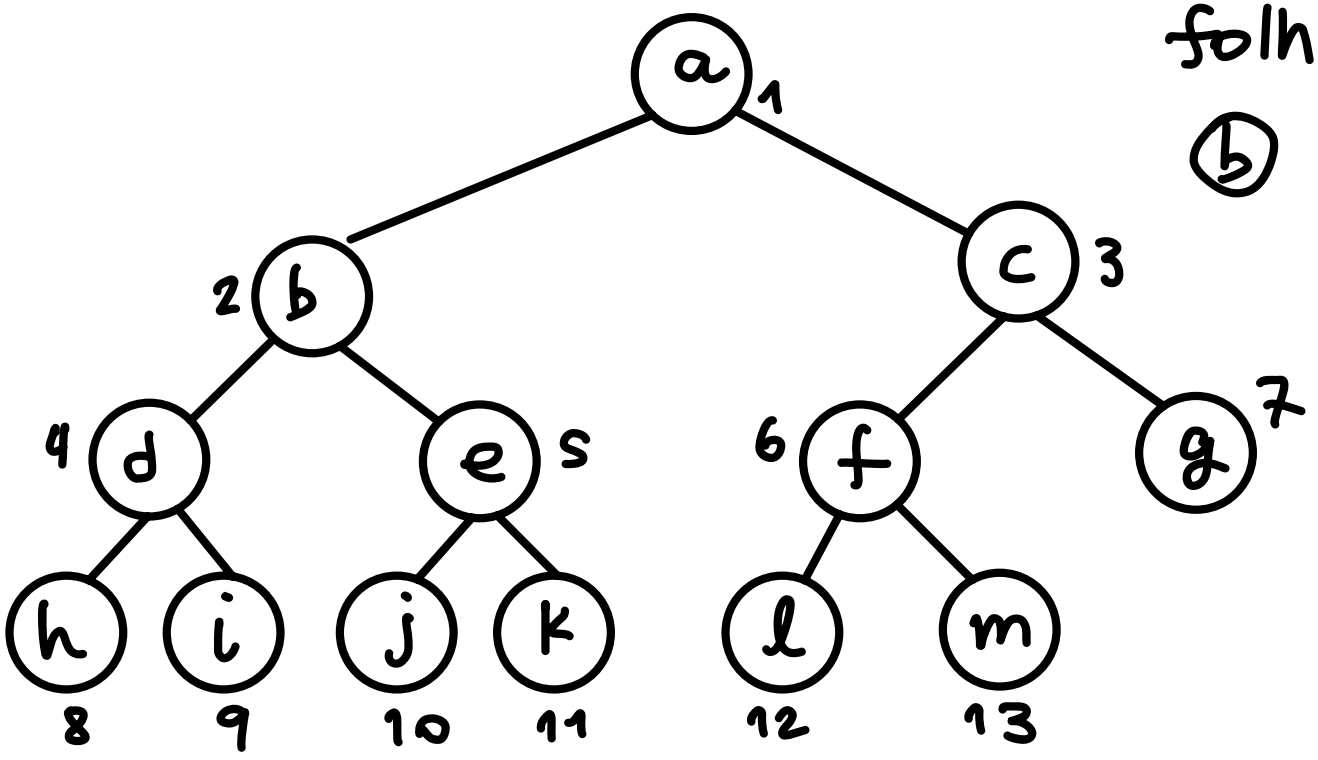


- O vértice i fica no na altura $\lfloor \lg(n/i) \rfloor$
- A altura da árvore é a altura da raiz
- As **folhas** são os nós sem filhos, no caso do heap eles correspondem aos nós de altura 0
- Há no máximo $\lceil n/2^{h+1} \rceil$ nós de altura h

- Em um MIN-HEAP, o valor do pai de ser <= aos valor de seus filhos
- Note que isso implica que o **mínimo** dentre os valores em um MIN-HEAP está em A[1]



LISTA 9A — Q1



ⓐ Indique as folhas

ⓑ Qual a altura de a?

ⓒ Qual o nível de f?

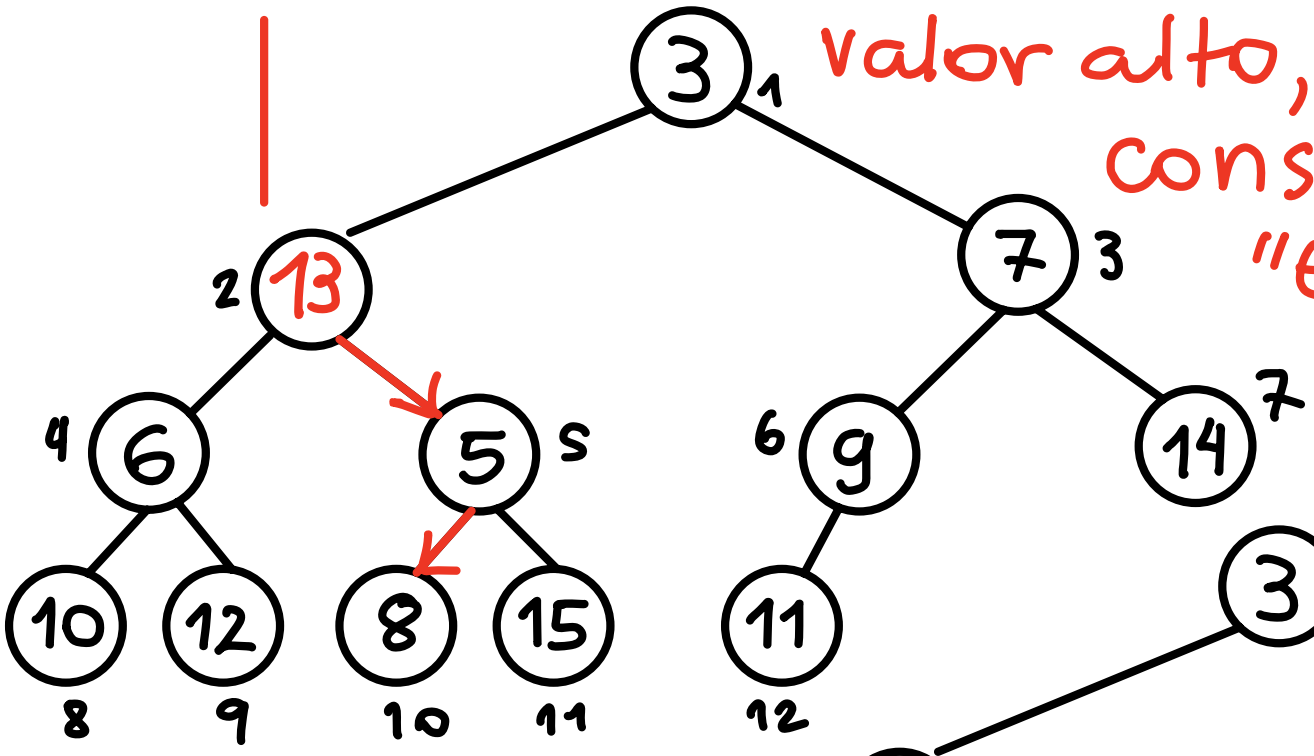
A

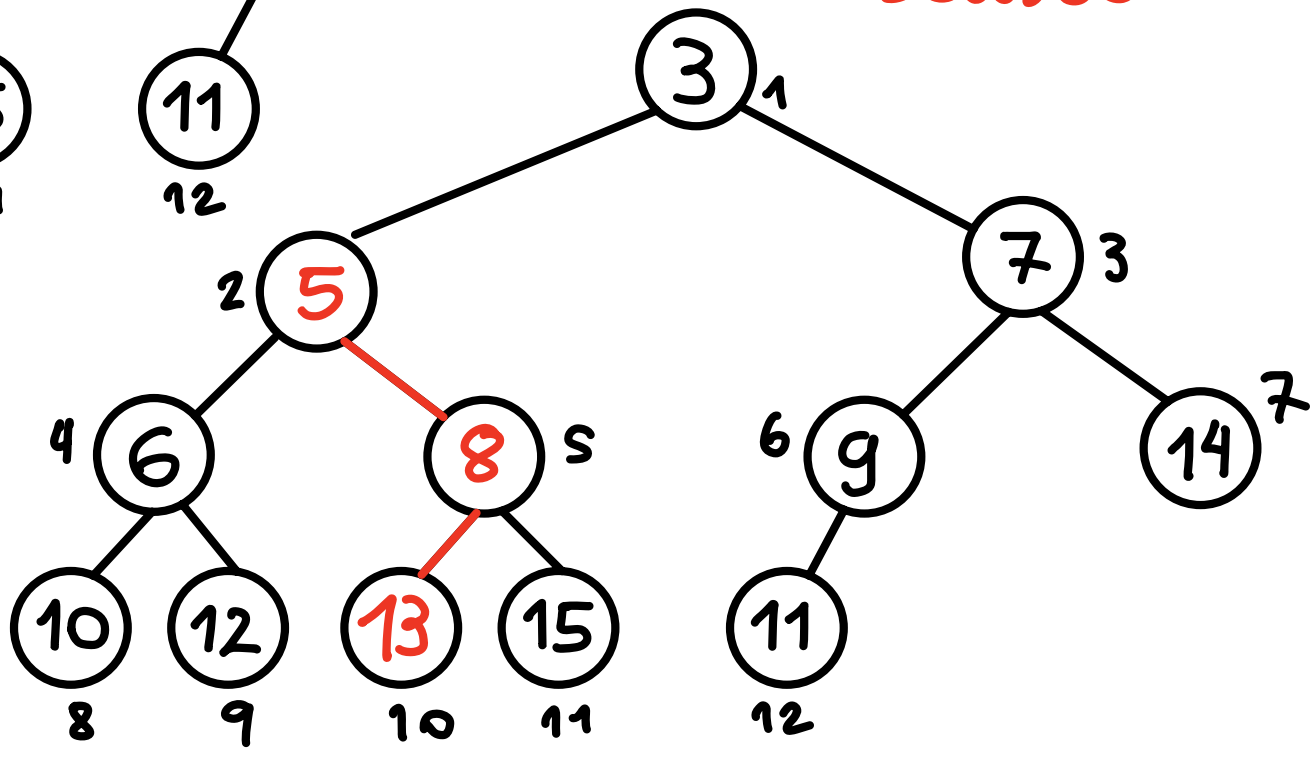
ⓓ se $l = 5$

$m = 7$

$c = 2$, digite um possível valor para f

se um elemento está errado por ter valor alto, podemos consertar "empurrando" o nó para "baixo"





```
CORRIGE-DESCENDO(A, n, i)
j = i
enquanto 2j ≤ n :  # tem filho
    e = 2j, d = 2j + 1, t = 0
    se A[e] < A[j] :
        t = e
    se d ≤ n :
        se A[d] < A[e] e A[d] < A[j] :
            t = d
    se t = 0 :
        break
    senão :
        A[t] ↔ A[j], j ← t
```

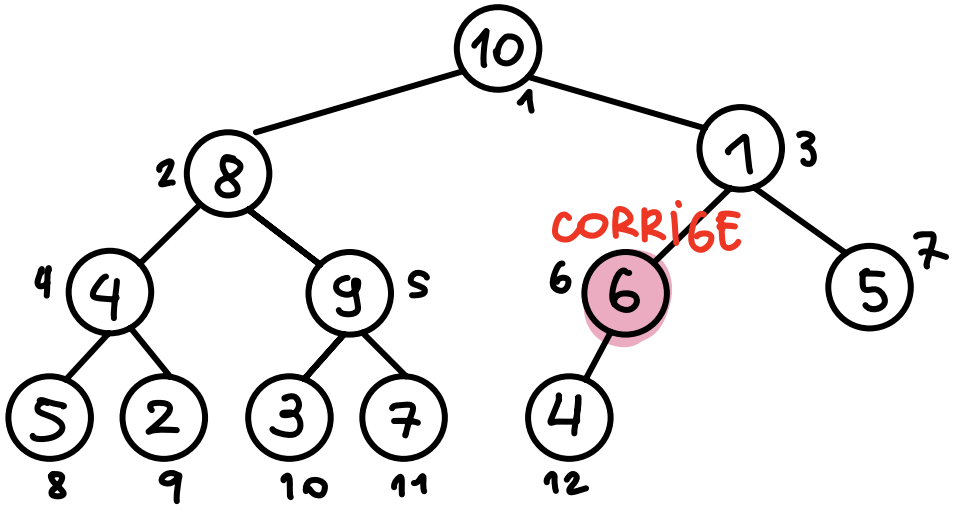
Tempo de execução

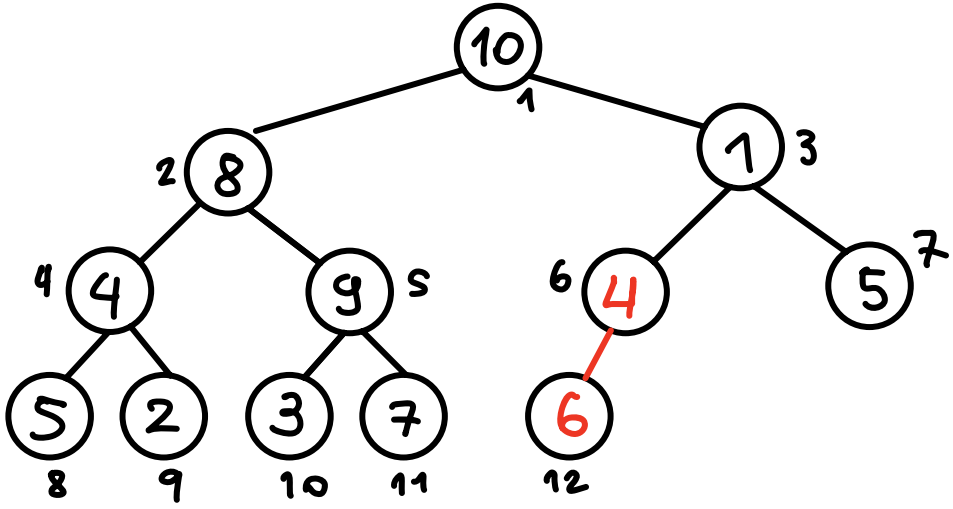
$O(\lg(n/i))$

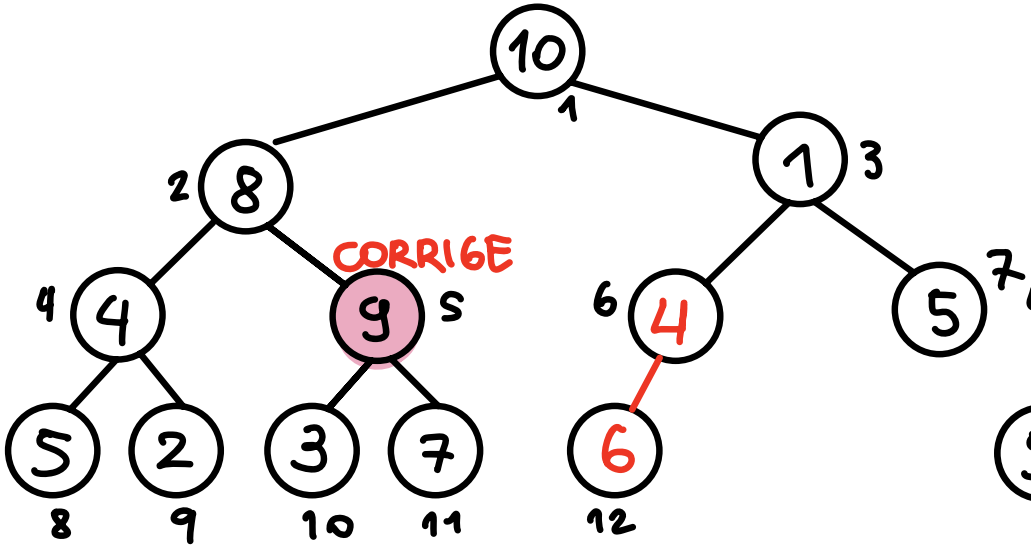
altura de i

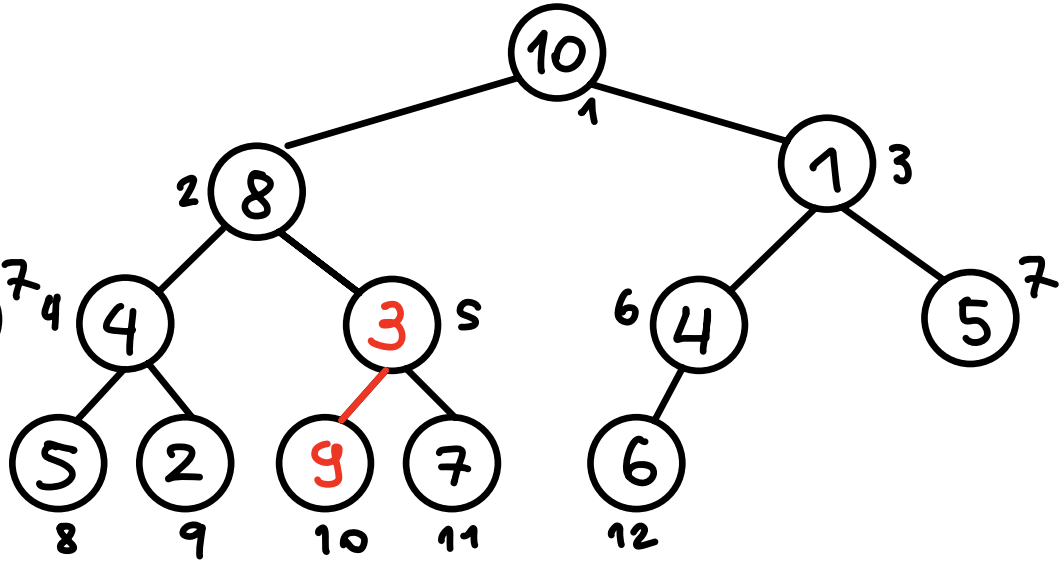
- Note que para i > n/2 não tem filhos
- Para transformar um vetor A em um heap, basta chamarmos CORRIGE-DESCENDO para i de n/2 até 1:

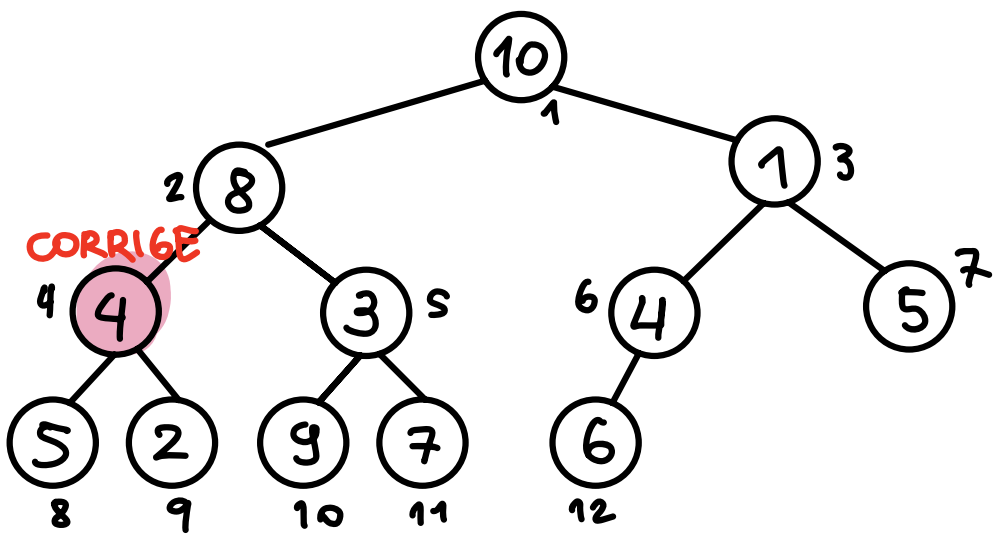
```
CONSTRÓI-MIN-HEAP(A,n):
  para i de ⌊n/2⌋ a 1:
      CORRIGE-DESCENDO(A,n,i)
```

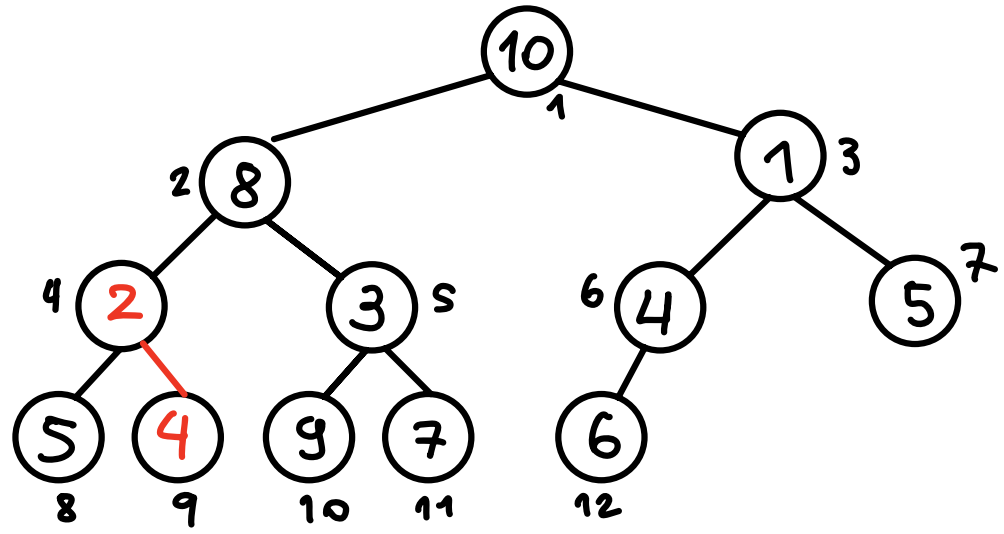

CORRIGE

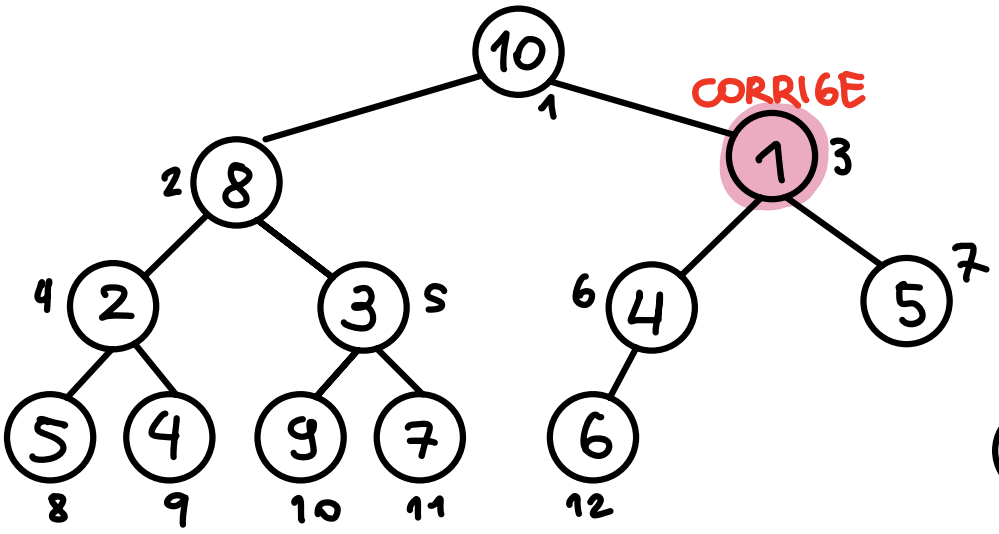


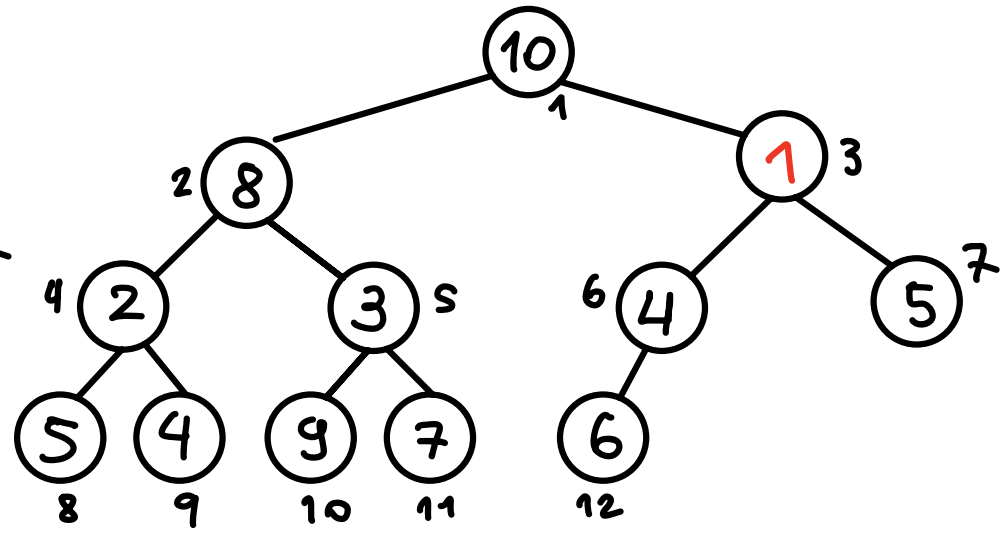

CORRIGE

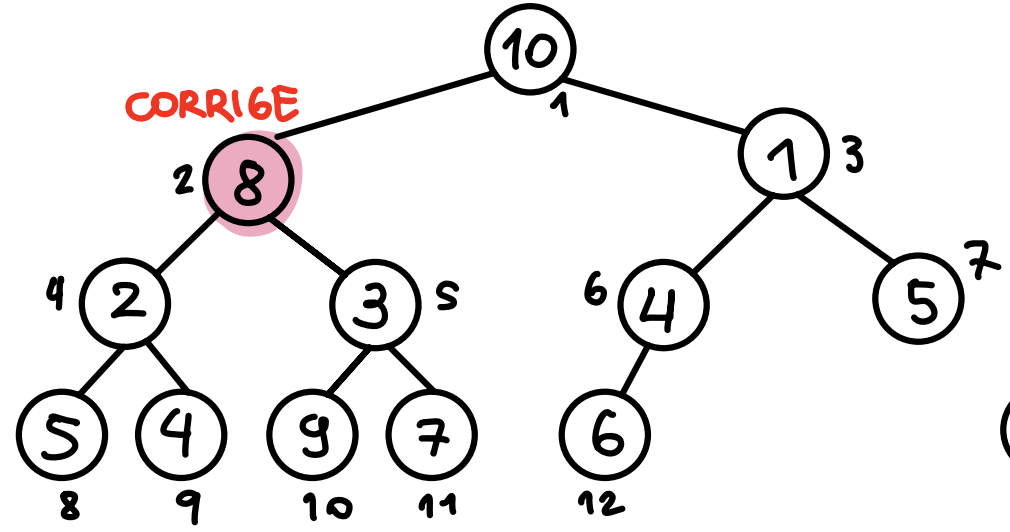


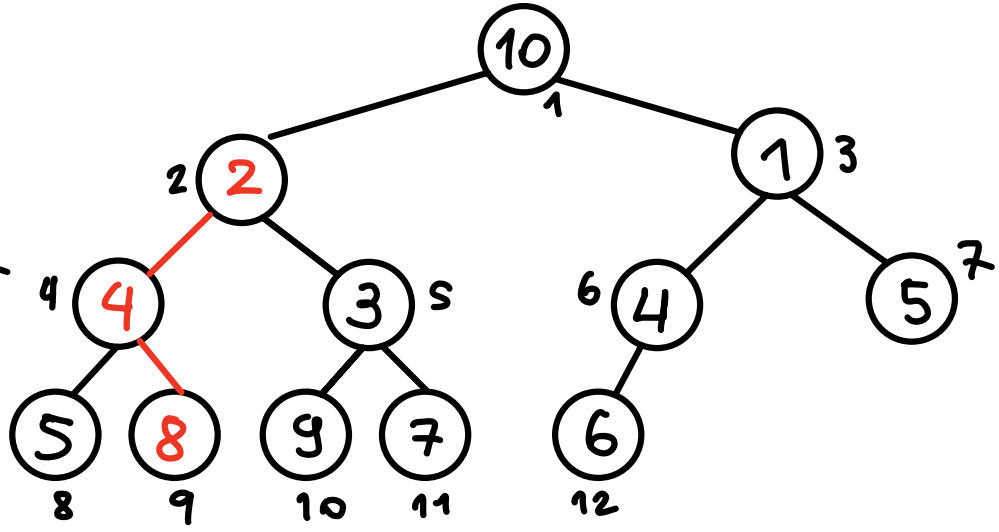

CORRIGE

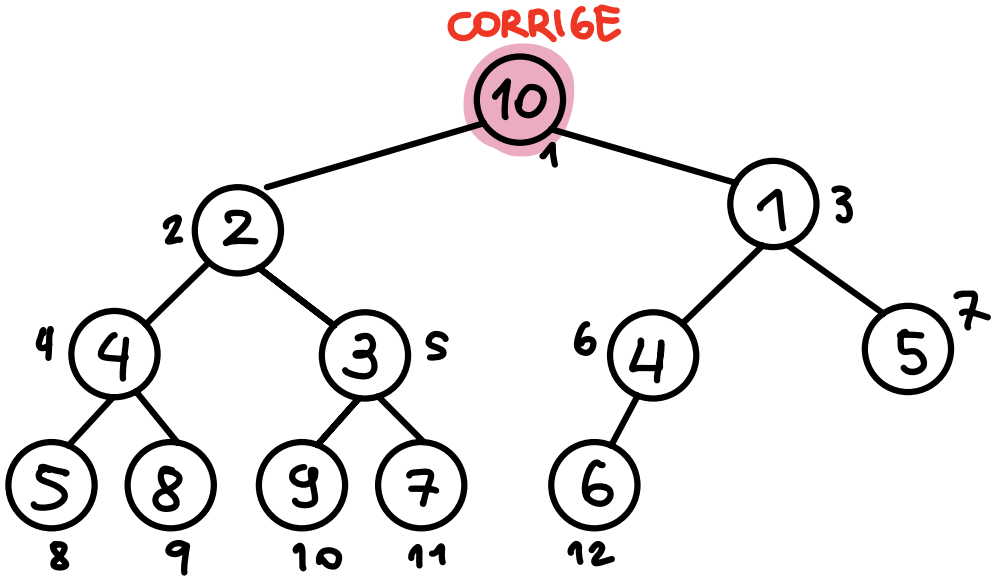


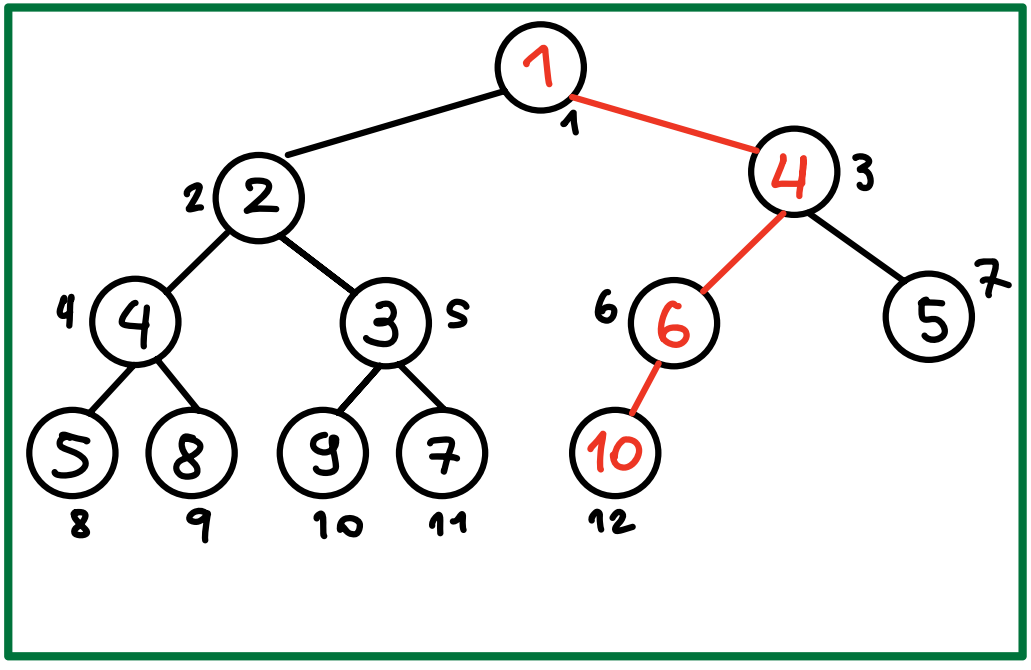

CORRIGE



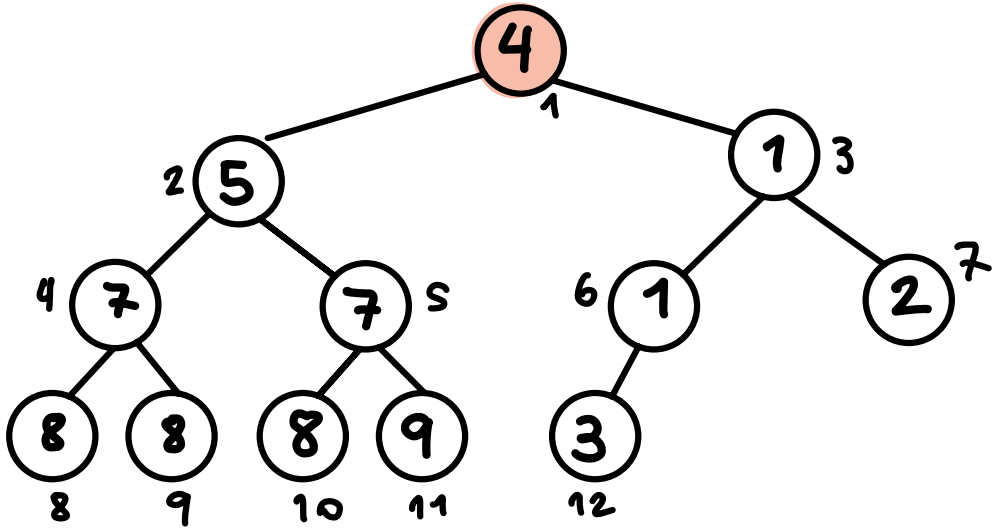

CORRIGE
10
1
8
2
1
3
2
4
3
5
4
6
5
7
5
8
4
9
9
10
7
11
6
12


10
1
2
2
1
3
4
4
3
5
4
6
5
7
5
8
8
9
9
10
7
11
6
12


CORRIGE
10
1
2
2
1
3
4
4
3
5
4
6
5
7
5
8
8
9
9
10
7
11
6
12


1
1
2
2
4
3
4
4
3
5
6
6
5
7
5
8
8
9
9
10
7
11
10
12

LISTA 9A — Q2



Rode CORRIGE-DESCENDO(A, n, 1)

```
CONSTRÓI-MIN-HEAP(A,n):
  para i de ⌊n/2⌋ a 1:
      CORRIGE-DESCENDO(A,n,i)
```

$O(\lg(n/i))$ para cada i

$$\sum_{i=1}^{n/2} \lg(n/i) = \lg\Big(\underbrace{\frac{n}{1}\cdot\frac{n}{2}\cdot\frac{n}{3}\cdot\ldots\cdot\frac{n}{n/2}}_{A}\Big)$$

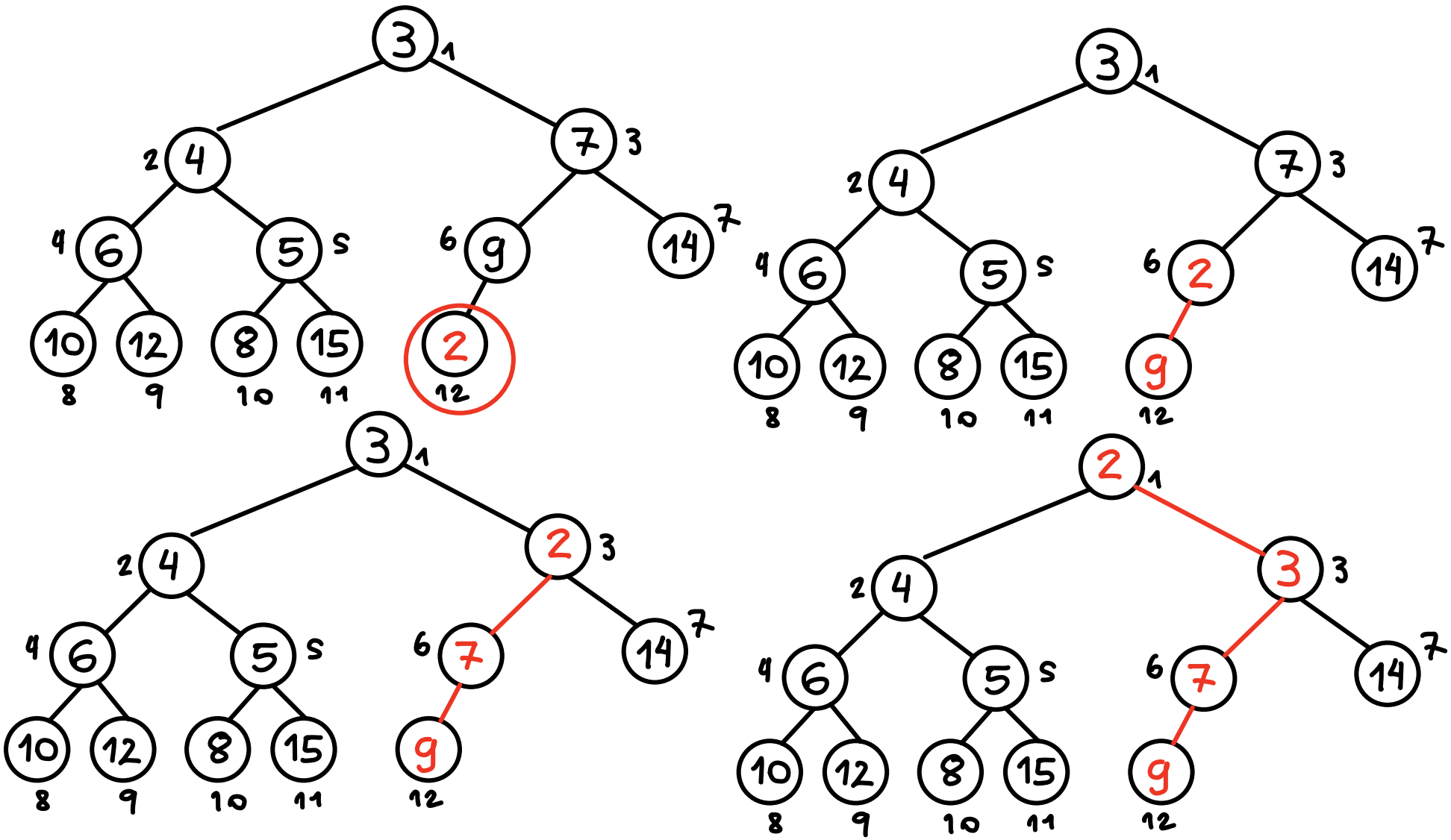
$$A = \frac{n^{n/2}}{(n/2)!} \longrightarrow \text{Stirling } x! \sim \sqrt{2\pi x}\left(\frac{x}{e}\right)^{x}$$

$$\frac{n^{n/2}}{(n/2)!} \leq \frac{n^{n/2}}{\left(\frac{n/2}{e}\right)^{n/2}} = (2e)^{n/2}$$

$$\lg\left((2e)^{n/2}\right) = \frac{n}{2}\lg(2e) = O(n)$$

Tempo de execução : $O(n)$
(pode ser mais rápido)

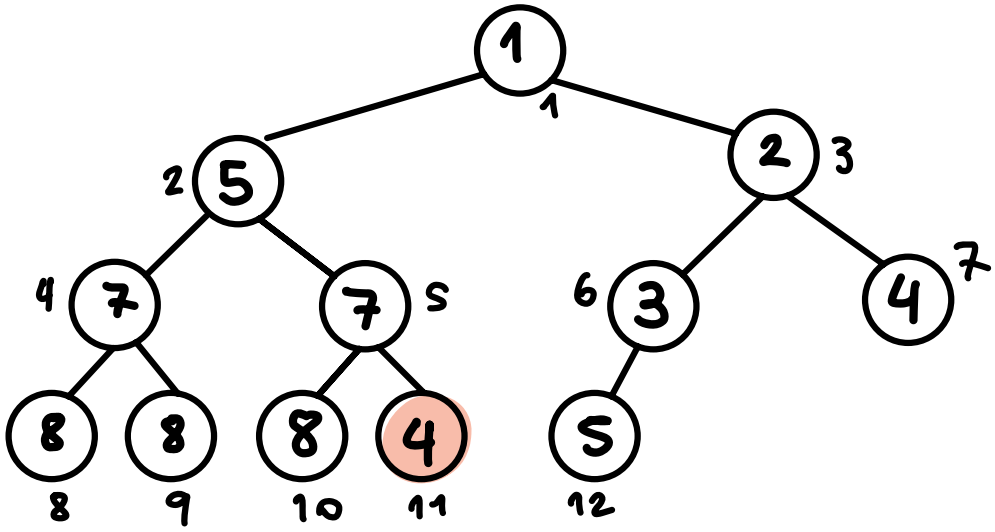
se um elemento está errado por ter valor <u>baixo</u> podemos consertar "empurrando" o nó para "cima"



```
CORRIGE-SUBINDO(A,m)
  pai = ⌊m/2⌋  filho = m
  enquanto pai ≥ 1:
      se A[pai] > A[filho]:
          A[pai] ↔ A[filho]:
          filho = pai
          pai = ⌊pai/2⌋
      senão:
          break
```

$\Theta(\lg(m))$

LISTA 9A — Q3



Rode CORRIGE-SUBINDO(A, 11)

MIN-HEAP

CRIA(N): cria um heap com capacidade de armazenamento N. Retorna o heap criado

INSERE(v): insere um elemento com valor v

REMOVE(): remove o elemento de **menor** valor

A

n = número de elementos no array

N = capacidade

```
CRIA(N)
  A ← um vetor de tamanho N
  n = 0
```

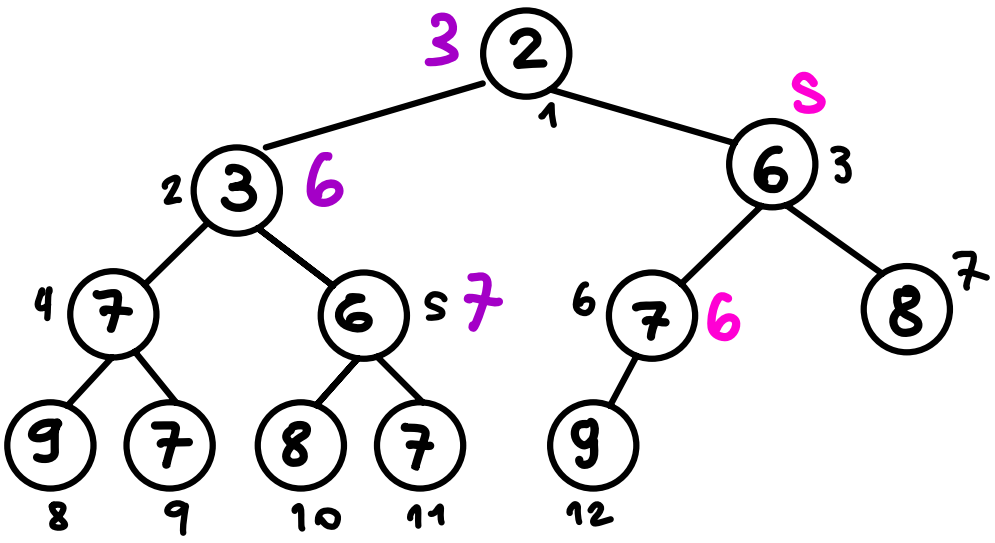
```
REMOVE( )
  v = A[1]
  u = A[n] # última posição
  n--
  A[1]=u
  CORRIGE_DESCENDO(A,n,1)
  retorne v
```

$O(\lg n)$

```
INSERE(v)
  n++
  A[n]=v
  CORRIGE_SUBINDO(A,n,n)
```

$O(\lg n)$

LISTA 9A — Q4



INSIRA(5)

REMOVE()

- Veja uma maneira simples de ordenar um vetor A usando um MIN-HEAP

```
ORDENA(A,n)
  H ← CRIA(n)
  para i de 1 a n:
      H.INSERE(A[i])
  para i de 1 a n:
      A[i] = H.REMOVE()
```

- O HeapSort usa um heap, mas no próprio vetor A. Interessantemente, é usado um **MAX-HEAP** e não um **MIN-HEAP**

MAX-HEAP

CRIA(N): cria um heap com capacidade de armazenamento N. Retorna o heap criado

INSERE(v): insere um elemento com valor v

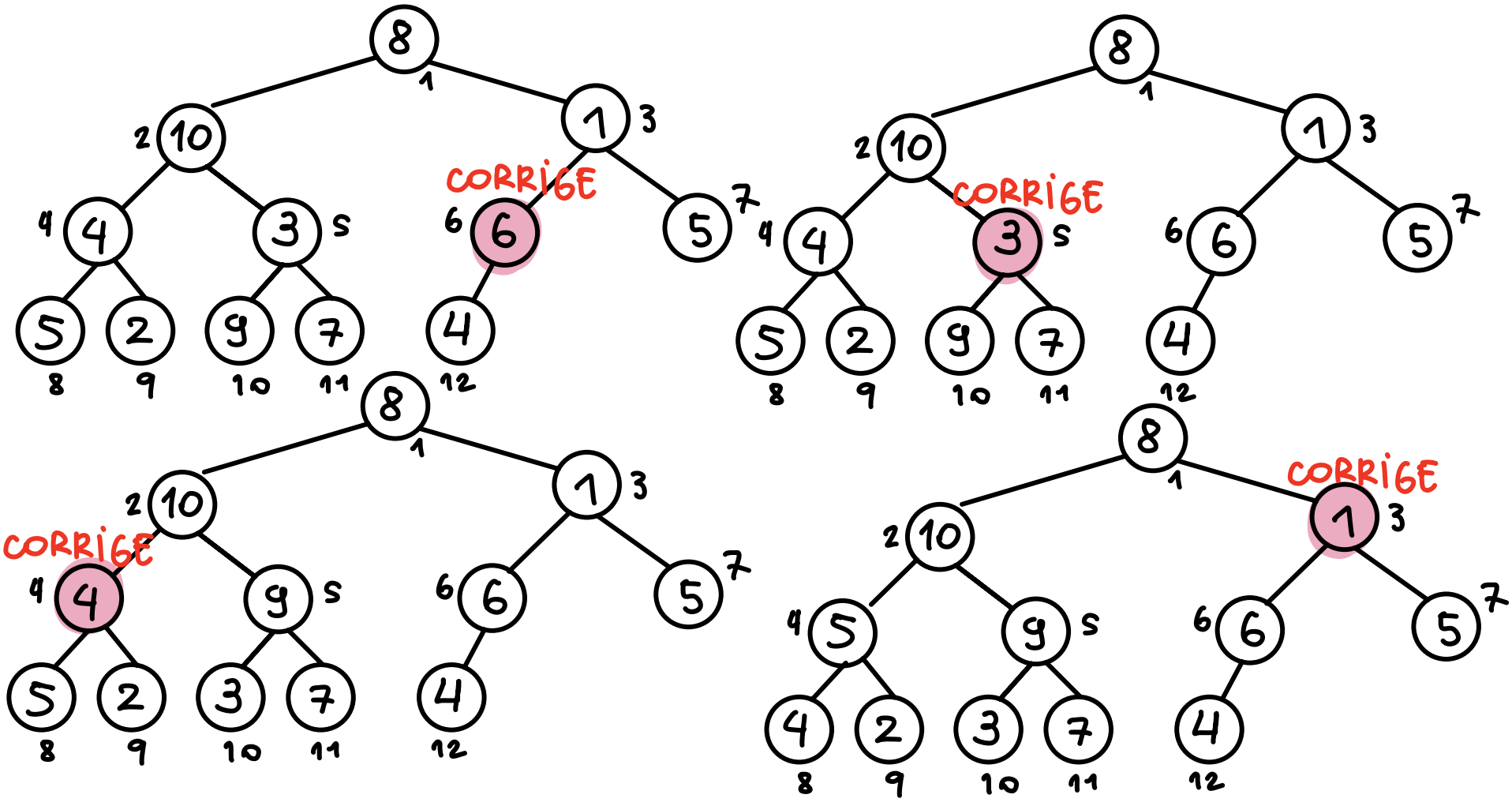
REMOVE(): remove o elemento de **maior** valor

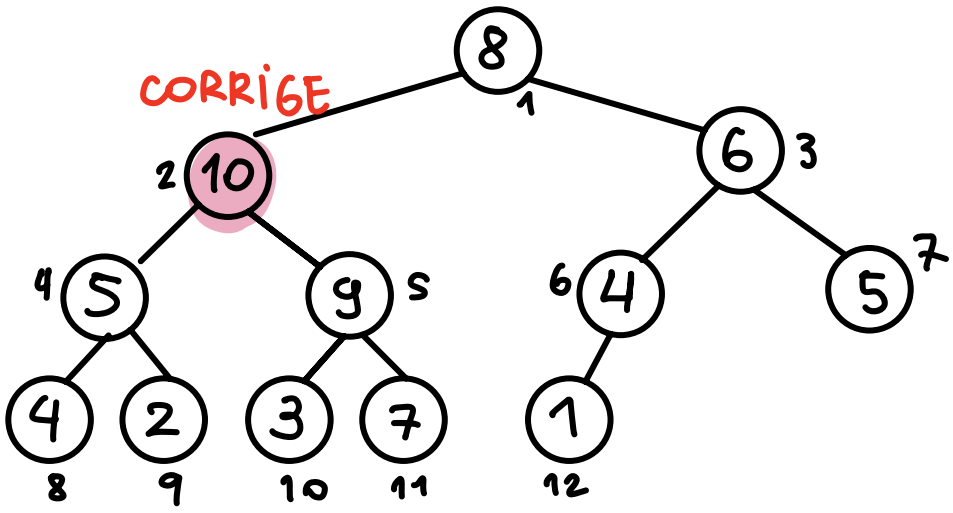
- A implementação do MAX-HEAP é muito parecida com o MIN-HEAP
- A propriedade que queremos manter é que se i é pai de j, então A[i] >= A[j]
- Dessa forma, o máximo fica na posição 1
- Para que adaptar a implementação basta trocar alguns sinais <= para >=, e vice-versa

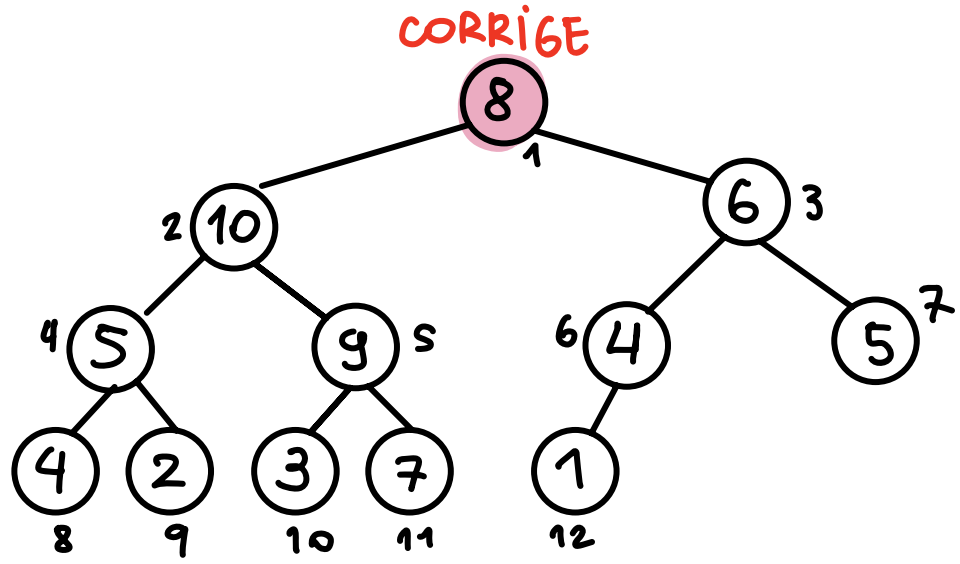
- O HeapSort começa chamando CONSTRÓI-**MAX**-HEAP(A,n). Vamos ver como fica:

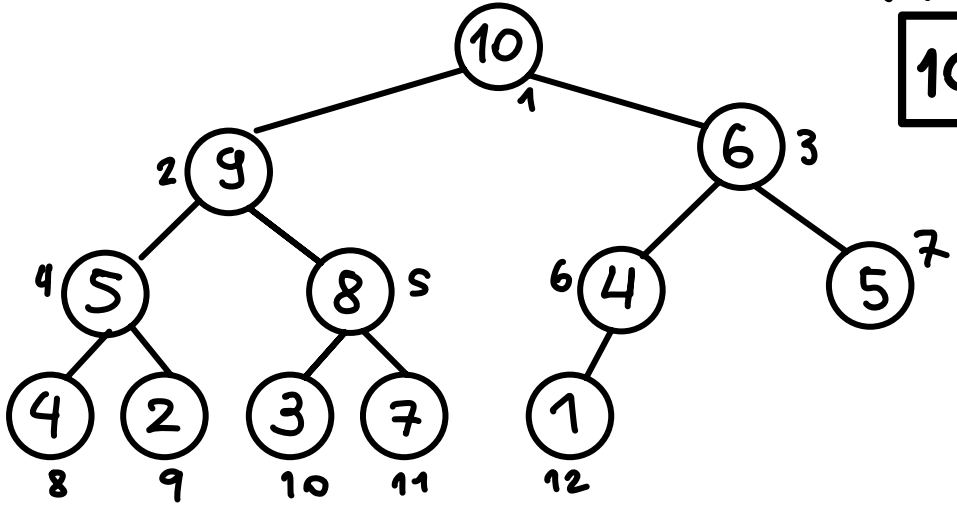
A

8	10	1	4	3	6	5	5	2	9	7	4









A

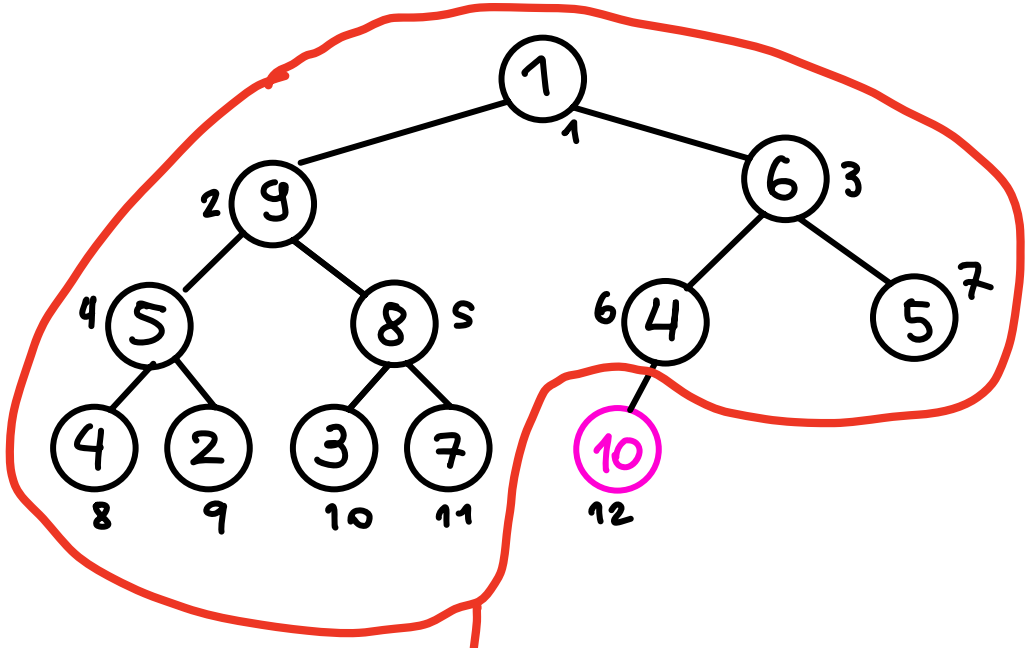
10	9	6	5	8	4	5	4	2	3	7	1

Não parece que ajudou...

A

10	9	6	5	8	4	5	4	2	3	7	1

Vamos trocar o 10 com o 1



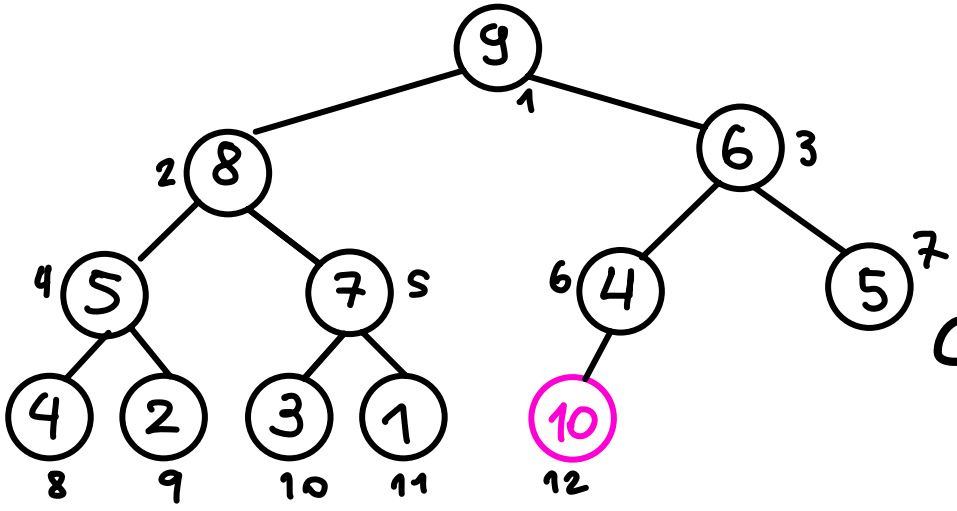
Pelo menos sabemos que o máximo foi para a última posição

Não é um MAX-HEAP pois 1 está na posição errada

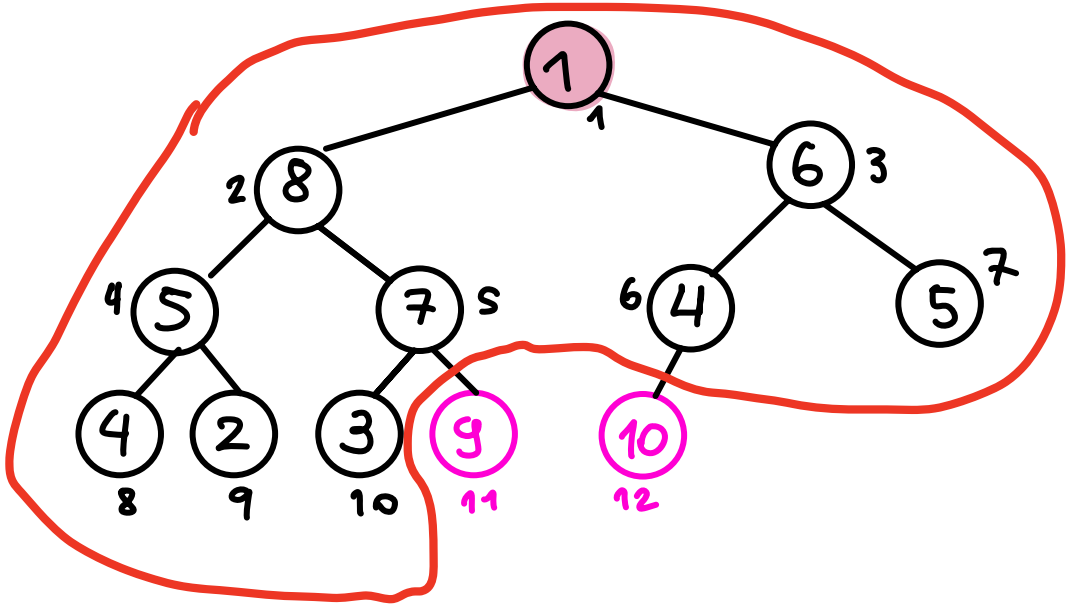
Vamos chamar CORRIGE-DESCENDO para A até 11 (versão MAX-HEAP)

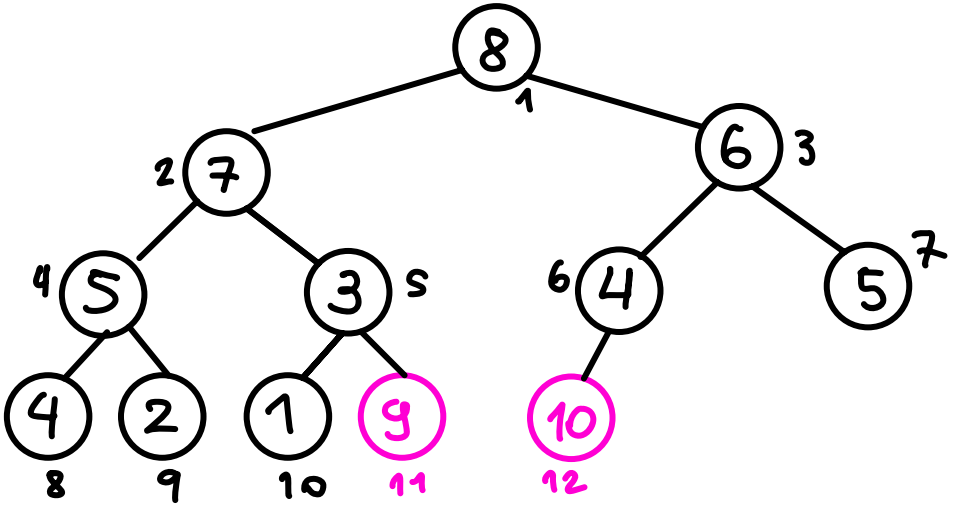
A

9	8	6	5	7	4	5	4	2	3	1	10



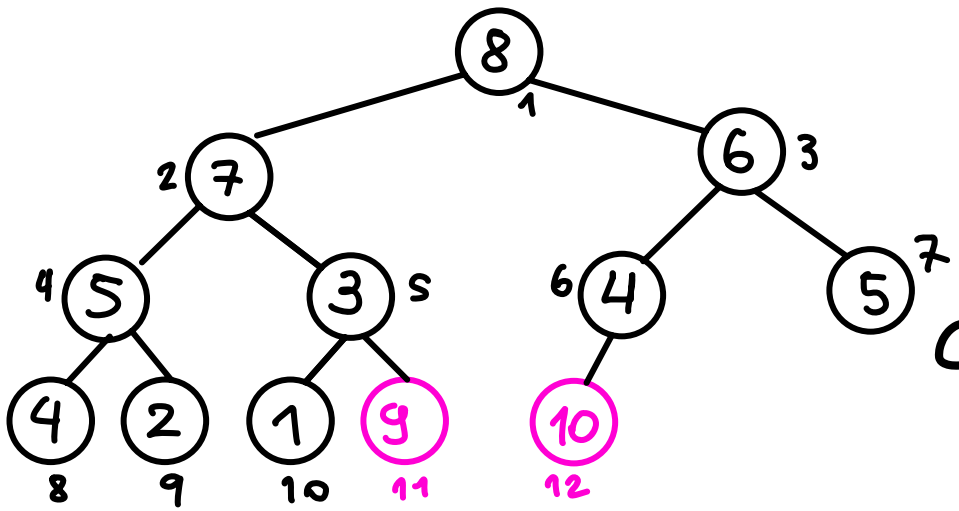
Agora trocamos o A[1]=9 com o A[11]=1 e chamamos CORRIGE-DESCENDO(A, 10, 1)



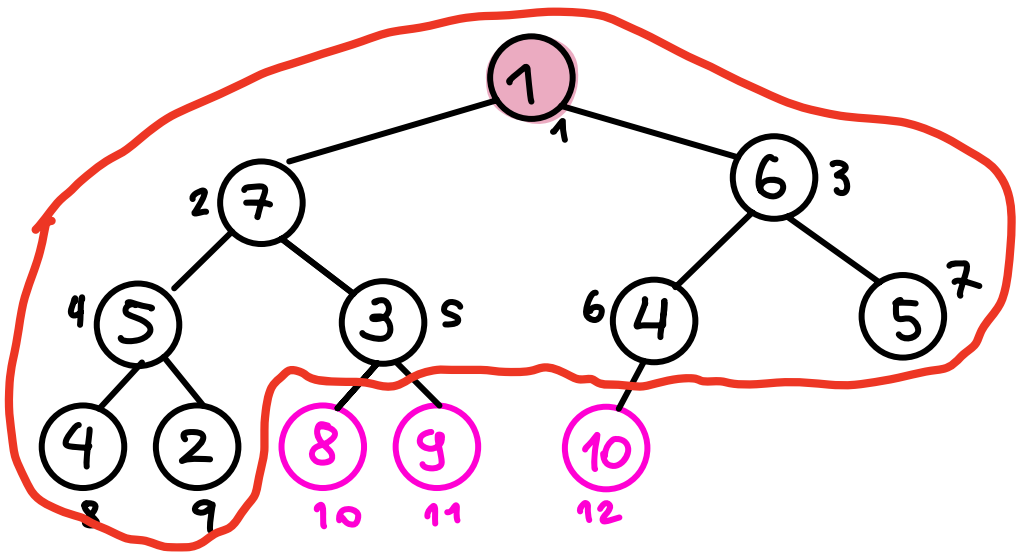


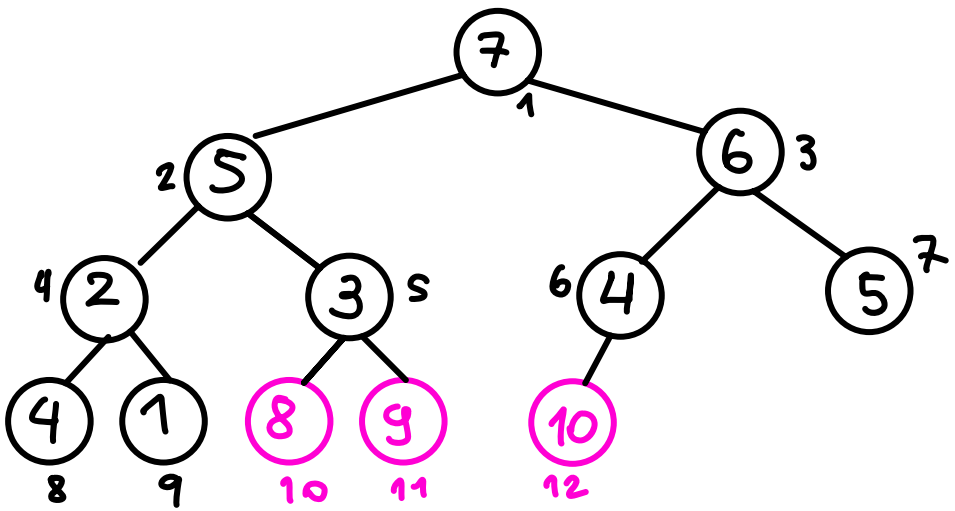
A

8	7	6	5	3	4	5	4	2	1	9	10



Agora trocamos o A[1] = 8 com o A[10] = 1 e chamamos CORRIGE-DESCENDO(A, 9, 1)

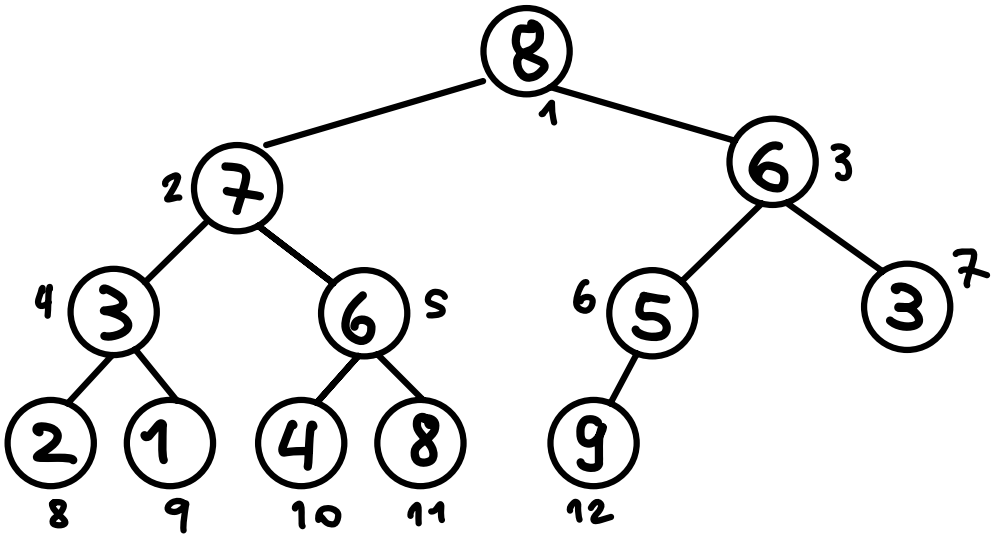




```
HEAPSORT(A, n)
    CONSTRÓI_MAX_HEAP(A, n)                    O(n)
    para i de n a 2:
        A[1] ↔ A[i]
        CORRIGE_DESCENDO(A, i-1, 1)            O(lg n)
```

Tempo de execução: $O(n \lg n)$

LISTA 9A — Q5



Rode o laço
para $i = n-2$